



EXPERIMENT – 2

NAME: Sam Fawaz Hadi AL_Basara
BRANCH: CSE-AIML (CS221)
SEMESTER: 5TH
SUBJECT: Full Stack - II

UID: 24BAI70236
SECTION/GROUP: 24AIT_NTPP 2[A]
DATE OF PERFORMANCE: 17/07/2026
SUBJECT CODE: 24CSP-337

EXPERIMENT 2.1

1. AIM

To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data.

2. OBJECTIVE

1. To understand the concept of global state management
2. To implement Redux Toolkit for scalable state handling
3. To design a normalized state structure
4. To manage asynchronous data flows (optionally via mock APIs)

3. SOFTWARE REQUIREMENT

1. Node.js & npm
2. React.js
3. Redux Toolkit
4. React-Redux
5. Code Editor (VS Code)

4. THEORY

As frontend applications grow in complexity, managing state across multiple components becomes increasingly challenging. Traditional approaches such as prop drilling lead to tightly coupled components and reduced maintainability.

Redux provides a centralized store that acts as a single source of truth, enabling predictable state transitions through actions and reducers. Redux Toolkit simplifies Redux implementation by reducing boilerplate and providing built-in utilities for common tasks.

State normalization is an important concept where data is stored in a structured format (similar to relational databases), reducing redundancy and improving performance. This is especially useful when managing collections such as posts or users.

Additionally, Redux supports handling asynchronous operations, enabling integration with APIs or simulated mock data sources.

5. IMPLEMENTATION/PROCEDURE

1. Open Visual Studio Code.
2. Create a React application using Create React App.
3. Install Redux Toolkit and React-Redux libraries.
4. Configure the Redux Store using `configureStore()`.
5. Create a Redux Slice (`postsSlice.js`) with an initial state.
6. Define reducers for Add, Update, and Delete operations.
7. Wrap the application with the Redux Provider.
8. Create the `PostList` component.
9. Use `useSelector()` to retrieve posts from the Redux Store.
10. Use `useDispatch()` to dispatch Redux actions.

11. Implement CRUD operations: Add Post, Update Post, Delete Post
12. Design the interface using CSS.
13. Run the application using: npm start
14. Verify all CRUD operations.

6. CODE

Store. Jsx

```
import { configureStore } from '@reduxjs/toolkit';
import postsReducer from './postsSlice';
import platformsReducer from './platformsSlice';
```

```
export const store = configureStore({
  reducer: {
    posts: postsReducer,
    platforms: platformsReducer,
  },
});
```

```
export default store;
```

PostsSlice. Js

```
import { createSlice } from '@reduxjs/toolkit';
```

```
const initialState = {
  posts: [
    {
      id: 'post-1',
      title: ' Launching Centralized State Architecture',
      content: 'Exploring how Redux Toolkit simplifies global state management in modern scalable frontend applications. Single store, predictable state updates, zero prop drilling!',
      platforms: ['linkedin', 'twitter'],
      status: 'Published',
      tags: ['ReduxToolkit', 'ReactJS', 'WebDev'],
      createdAt: new Date(Date.now() - 3600000 * 2).toISOString(),
      author: 'Sam Fawaz Hadi Ali AL-Basara (24BAI70236)'
    },
    {
      id: 'post-2',
      title: ' AI & ML Full Stack Integration Insights',
      content: 'Combining deep learning models with React state management provides real-time monitoring of inference pipelines and dataset tracking.',
      platforms: ['twitter', 'instagram'],
      status: 'Draft',
      tags: ['AIML', 'FullStack', 'StateManagement'],
      createdAt: new Date(Date.now() - 3600000 * 5).toISOString(),
      author: 'Sam Fawaz Hadi Ali AL-Basara (24BAI70236)'
    }
  ]
};
```

```
const postsSlice = createSlice({
  name: 'posts',
  initialState,
  reducers: {
    addPost: (state, action) => {
      const newPost = {
        id: `post-${Date.now()}`,
        createdAt: new Date().toISOString(),
        author: 'Sam Fawaz Hadi Ali AL-Basara (24BAI70236)',
        status: action.payload.status || 'Draft',
        tags: action.payload.tags || [],
        platforms: action.payload.platforms || ['twitter'],
        ...action.payload
      };
      state.posts.unshift(newPost);
    },
    deletePost: (state, action) => {
      state.posts = state.posts.filter((post) => post.id !== action.payload);
    },
    toggleStatus: (state, action) => {
      const post = state.posts.find((p) => p.id === action.payload);
      if (post) {
        post.status = post.status === 'Published' ? 'Draft' : 'Published';
      }
    },
    updatePost: (state, action) => {
      const index = state.posts.findIndex((p) => p.id === action.payload.id);
      if (index !== -1) {
        state.posts[index] = { ...state.posts[index], ...action.payload };
      }
    }
  }
});

export const { addPost, deletePost, toggleStatus, updatePost } = postsSlice.actions;
export default postsSlice.reducer;
```

Index. Js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";

import { Provider } from "react-redux";
import { store } from "./app/store";

const root = ReactDOM.createRoot(document.getElementById("root"));
```

```
    root.render(  
      <Provider store={store}>  
        <App />  
      </Provider>  
    );
```

App. Jsx

```
import React from "react";  
import Header from "../components/Header";  
import PostComposer from "../components/PostComposer";  
import DraftList from "../components/DraftList";  
import StateInspector from "../components/StateInspector";
```

```
function App() {  
  return (  
    <div className="app-viewport">  
      <div className="app-container">  
        <Header />  
  
        <main className="app-main-grid">  
          <section className="composer-column">  
            <PostComposer />  
          </section>  
  
          <section className="feed-column">  
            <DraftList />  
          </section>  
        </main>  
  
        <footer className="app-footer">  
          <StateInspector />  
          <div className="footer-meta">  
            <p>© 2026 Sam Fawaz Hadi Ali AL-Basara — UID: 24BAI70236 | Full Stack-II (24CSP-337)  
Centralized State Lab</p>  
          </div>  
        </footer>  
      </div>  
    );  
  }  
  
  export default App;
```

postList.js

```
import React, { useState } from "react";  
import { useSelector, useDispatch } from "react-redux";  
import {  
  addPost,  
  updatePost,
```

```
deletePost,
} from "../features/posts/postsSlice";

function PostList() {
  const [title, setTitle] = useState("");

  const posts = useSelector((state) => state.posts.items);
  const dispatch = useDispatch();

  const handleAddPost = () => {
    if (title.trim() === "") return;

    dispatch(
      addPost({
        id: Date.now(),
        title: title,
      })
    );

    setTitle("");
  };

  const handleUpdatePost = (post) => {
    const newTitle = prompt("Enter new title:", post.title);

    if (newTitle) {
      dispatch(
        updatePost({
          id: post.id,
          title: newTitle,
        })
      );
    }
  };

  const handleDeletePost = (id) => {
    dispatch(deletePost(id));
  };

  return (
    <div className="container">
      <h1>📁 Redux Content Manager</h1>

      <div className="input-section">
        <input
          type="text"
          placeholder="Enter Post Title..."
          value={title}
          onChange={(e) => setTitle(e.target.value)}
        />
      </div>
    </div>
  );
}
```



```
    />

    <button onClick={handleAddPost}>Add Post</button>
  </div>

  {posts.length === 0 ? (
    <p>No Posts Available</p>
  ) : (
    posts.map((post) => (
      <div className="card" key={post.id}>
        <span>{post.title}</span>

        <div>
          <button
            className="edit-btn"
            onClick={() => handleUpdatePost(post)}
          >
            Edit
          </button>

          <button
            className="delete-btn"
            onClick={() => handleDeletePost(post.id)}
          >
            Delete
          </button>
        </div>
      </div>
    ))
  )}
</div>
);
}
```

```
export default PostList;
```

App.cs

```
body {
  margin: 0;
  background: linear-gradient(135deg, #667eea, #764ba2);
  font-family: Arial, sans-serif;
}

.container {
  width: 700px;
  margin: 40px auto;
  background: white;
  padding: 30px;
  border-radius: 15px;
```



```
    box-shadow: 0 10px 25px rgba(0,0,0,0.2);  
  }
```

```
h1 {  
  text-align: center;  
  color: #4a148c;  
}
```

```
.input-section {  
  display: flex;  
  gap: 10px;  
  margin-bottom: 25px;  
}
```

```
input {  
  flex: 1;  
  padding: 12px;  
  border-radius: 8px;  
  border: 2px solid #667eea;  
  font-size: 16px;  
}
```

```
button {  
  padding: 10px 18px;  
  border: none;  
  border-radius: 8px;  
  cursor: pointer;  
  color: white;  
  font-weight: bold;  
}
```

```
button:hover {  
  opacity: 0.9;  
}
```

```
.input-section button {  
  background: #4caf50;  
}
```

```
.card {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background: #f4f6ff;  
  margin: 12px 0;  
  padding: 15px;  
  border-radius: 10px;  
  box-shadow: 0 3px 8px rgba(0,0,0,0.1);  
}
```

```
.edit-btn {
  background: #ff9800;
  margin-right: 10px;
}

.delete-btn {
  background: #f44336;
}

p {
  text-align: center;
  color: #666;
}
```

7. OUTPUT

The Redux Content Manager application was successfully implemented. Users were able to add new posts, edit existing posts, and delete posts through a centralized Redux Store. The interface updated automatically after every CRUD operation, demonstrating efficient state management using Redux Toolkit.

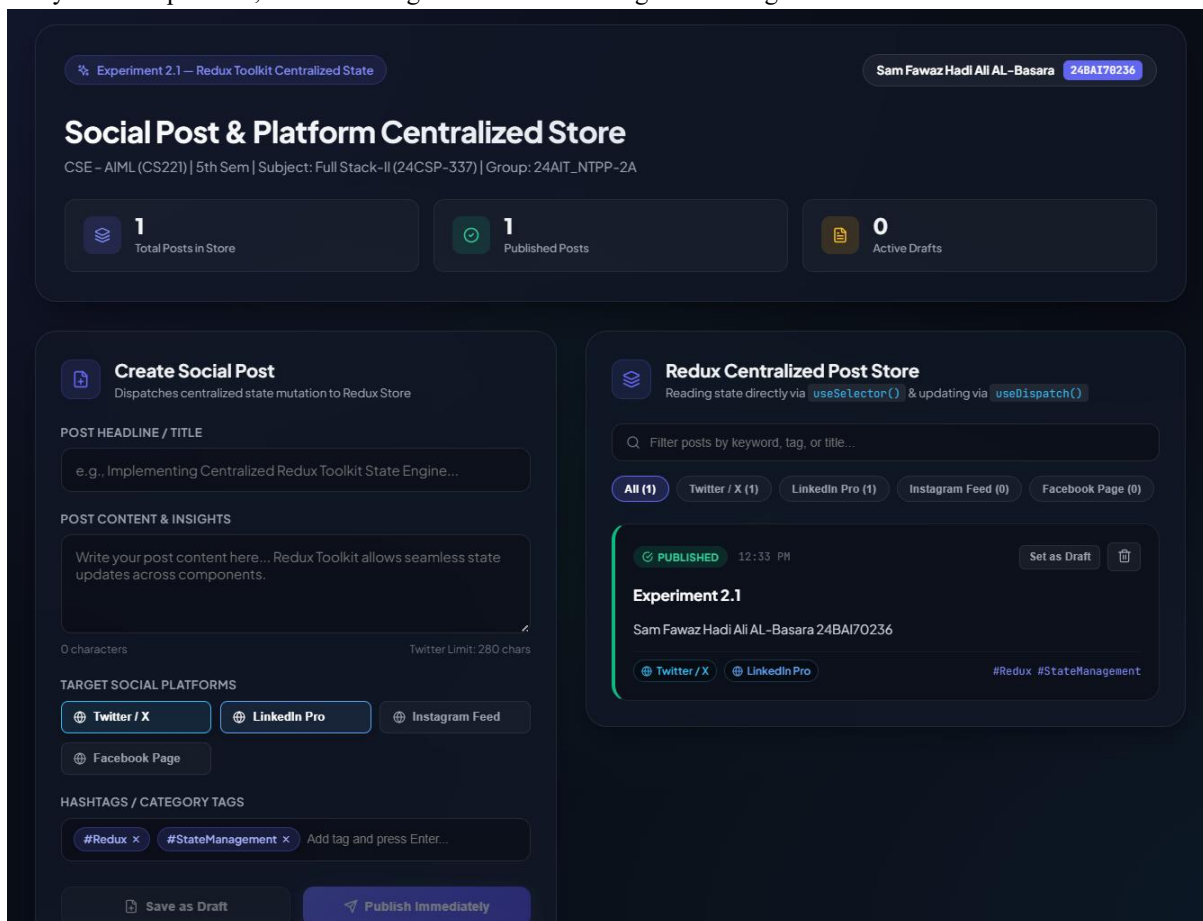


Fig: 2.1 Redux Content Manager application

8. LEARNING OUTCOMES

1. Understand derived state using Redux Selectors.



2. Implement memoized selectors using createSelector().
3. Optimize state retrieval from the Redux Store.
4. Reduce unnecessary recomputations using memoization.
5. Filter and search data efficiently.
6. Improve rendering performance in React applications.
7. Develop scalable and performance-optimized frontend applications using Redux Toolkit.

EXPERIMENT 2.2

1. AIM

To optimize state access and improve application performance using memoized selectors and efficient rendering strategies

2. OBJECTIVE

1. To understand the concept of derived state
2. To implement memoized selectors using Reselect
3. To reduce unnecessary component re-renders
4. To improve performance in large-scale applications

3. SOFTWARE REQUIREMENT

1. React.js
2. Redux Toolkit
3. Reselect (or built-in createSelector)
4. Code Editor

4. THEORY

In large-scale applications, inefficient state access and frequent re-renders can significantly degrade performance. Redux selectors are used to extract specific pieces of state; however, without optimization, they may recompute values unnecessarily.

Memoization is a technique used to cache computed results and return them only when input data changes. Libraries like Reselect provide memoized selectors that prevent redundant computations and reduce rendering overhead.

Derived state refers to data computed from the existing state rather than stored directly. By using selectors, applications can compute complex data efficiently without duplicating state.

Optimizing selectors and rendering behavior ensures that components update only when necessary, improving responsiveness and scalability.

5. IMPLEMENTATION/PROCEDURE

1. Create a new React application.
2. Install Redux Toolkit and React-Redux.
3. Configure the Redux Store.
4. Create a Posts Slice containing sample post data.
5. Create memoized selectors using createSelector().
6. Create selectors for: Total Posts Short Posts Long Posts
7. Create a Dashboard component.
8. Display all posts using useSelector().
9. Implement a search bar for filtering posts.
10. Implement filter buttons: All, Short, Long
11. Display statistics using memoized selectors.
12. Apply CSS styling for the dashboard.
13. Run the application using: npm start
14. Verify selector functionality and filtering

6. CODE

Store. Js

```
import { configureStore } from "@reduxjs/toolkit";  
import postsReducer from "../features/posts/postsSlice";
```

```
export const store = configureStore({
  reducer: {
    posts: postsReducer,
  },
});
```

PostsSlice. Js

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  items: [
    { id: 1, title: "React Basics" },
    { id: 2, title: "Redux Toolkit" },
    { id: 3, title: "JavaScript ES6" },
    { id: 4, title: "Node.js" },
    { id: 5, title: "MongoDB" },
    { id: 6, title: "Express.js" },
    { id: 7, title: "HTML & CSS" },
    { id: 8, title: "Full Stack Development" },
  ],
};

const postsSlice = createSlice({
  name: "posts",
  initialState,
  reducers: {},
});

export default postsSlice.reducer;

Selector.js
import { createSelector } from "@reduxjs/toolkit";

// Basic Selector
export const selectPosts = (state) => state.posts.items;

// Memoized Selector - Total Posts
export const selectTotalPosts = createSelector(
  [selectPosts],
  (posts) => posts.length
);

// Memoized Selector - Short Posts (Title length < 12)
export const selectShortPosts = createSelector(
  [selectPosts],
  (posts) => posts.filter((post) => post.title.length < 12)
);
```

```
// Memoized Selector - Long Posts (Title length >= 12)
export const selectLongPosts = createSelector(
  [selectPosts],
  (posts) => posts.filter((post) => post.title.length >= 12)
);
```

Dashboard.js

```
import React, { useState } from "react";
import { useSelector } from "react-redux";

import {
  selectPosts,
  selectTotalPosts,
  selectShortPosts,
  selectLongPosts,
} from "../features/posts/selectors";

function Dashboard() {
  const posts = useSelector(selectPosts);
  const totalPosts = useSelector(selectTotalPosts);
  const shortPosts = useSelector(selectShortPosts);
  const longPosts = useSelector(selectLongPosts);

  const [filter, setFilter] = useState("all");
  const [search, setSearch] = useState("");

  let filteredPosts = posts;

  if (filter === "short") {
    filteredPosts = shortPosts;
  }

  if (filter === "long") {
    filteredPosts = longPosts;
  }

  filteredPosts = filteredPosts.filter((post) =>
    post.title.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div className="dashboard">

      <h1>🏠 Redux Selector Dashboard</h1>

      <input
        type="text"
        placeholder="🔍 Search Posts..."
        value={search}>
```

```
      onChange={(e) => setSearch(e.target.value)}
    />

    <div className="stats">

      <div className="card blue">
        <h2>{totalPosts}</h2>
        <p>Total Posts</p>
      </div>

      <div className="card green">
        <h2>{shortPosts.length}</h2>
        <p>Short Posts</p>
      </div>

      <div className="card orange">
        <h2>{longPosts.length}</h2>
        <p>Long Posts</p>
      </div>

    </div>

    <div className="buttons">
      <button onClick={() => setFilter("all")}>All</button>

      <button onClick={() => setFilter("short")}>
        Short
      </button>

      <button onClick={() => setFilter("long")}>
        Long
      </button>
    </div>

    <div className="list">

      {filteredPosts.map((post) => (
        <div key={post.id} className="post">
          {post.title}
        </div>
      ))}

    </div>

  </div>
);
}

export default Dashboard;
```

App. Js

```
import './App.css';
import Dashboard from './components/Dashboard';

function App() {
  return (
    <div className="App">
      <Dashboard />
    </div>
  );
}

export default App;
```

App.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: "Segoe UI", sans-serif;
}

body {
  background: linear-gradient(135deg, #0f172a, #2563eb);
  min-height: 100vh;
}

.dashboard {
  width: 90%;
  max-width: 1000px;
  margin: 40px auto;
  background: #ffffff;
  border-radius: 18px;
  padding: 30px;
  box-shadow: 0 15px 35px rgba(0,0,0,0.25);
}

.dashboard h1 {
  text-align: center;
  color: #1e3a8a;
  margin-bottom: 25px;
}

.dashboard input {
  width: 100%;
  padding: 14px;
  border: 2px solid #2563eb;
  border-radius: 10px;
```



```
outline: none;
font-size: 16px;
margin-bottom: 25px;
}

.dashboard input:focus {
border-color: #1d4ed8;
box-shadow: 0 0 8px rgba(37,99,235,.3);
}

.stats {
display: flex;
gap: 20px;
justify-content: space-between;
margin-bottom: 25px;
}

.card {
flex: 1;
padding: 20px;
border-radius: 15px;
color: white;
text-align: center;
transition: 0.3s;
}

.card:hover {
transform: translateY(-5px);
}

.card h2 {
font-size: 34px;
margin-bottom: 8px;
}

.blue {
background: linear-gradient(135deg,#2563eb,#1d4ed8);
}

.green {
background: linear-gradient(135deg,#10b981,#059669);
}

.orange {
background: linear-gradient(135deg,#f59e0b,#ea580c);
}

.buttons {
display: flex;
```



```
justify-content: center;  
gap: 15px;  
margin-bottom: 30px;  
}
```

```
.buttons button {  
  background: #2563eb;  
  color: white;  
  border: none;  
  padding: 12px 28px;  
  border-radius: 10px;  
  cursor: pointer;  
  font-size: 15px;  
  font-weight: bold;  
  transition: .3s;  
}
```

```
.buttons button:hover {  
  background: #1d4ed8;  
  transform: scale(1.05);  
}
```

```
.list {  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
}
```

```
.post {  
  background: #f8fafc;  
  border-left: 6px solid #2563eb;  
  padding: 18px;  
  border-radius: 12px;  
  font-size: 18px;  
  font-weight: 500;  
  transition: .3s;  
}
```

```
.post:hover {  
  background: #eff6ff;  
  transform: translateX(5px);  
}
```

```
@media (max-width: 768px) {  
  
  .stats {  
    flex-direction: column;  
  }  
}
```

```
.buttons {
  flex-direction: column;
}

.buttons button {
  width: 100%;
}

}
```

7. OUTPUT

The Redux Selector Dashboard was successfully implemented using memoized selectors. The application displayed statistics such as total, short, and long posts, while allowing users to search and filter posts efficiently. Memoized selectors optimized state access and reduced unnecessary recomputations, resulting in improved application performance.

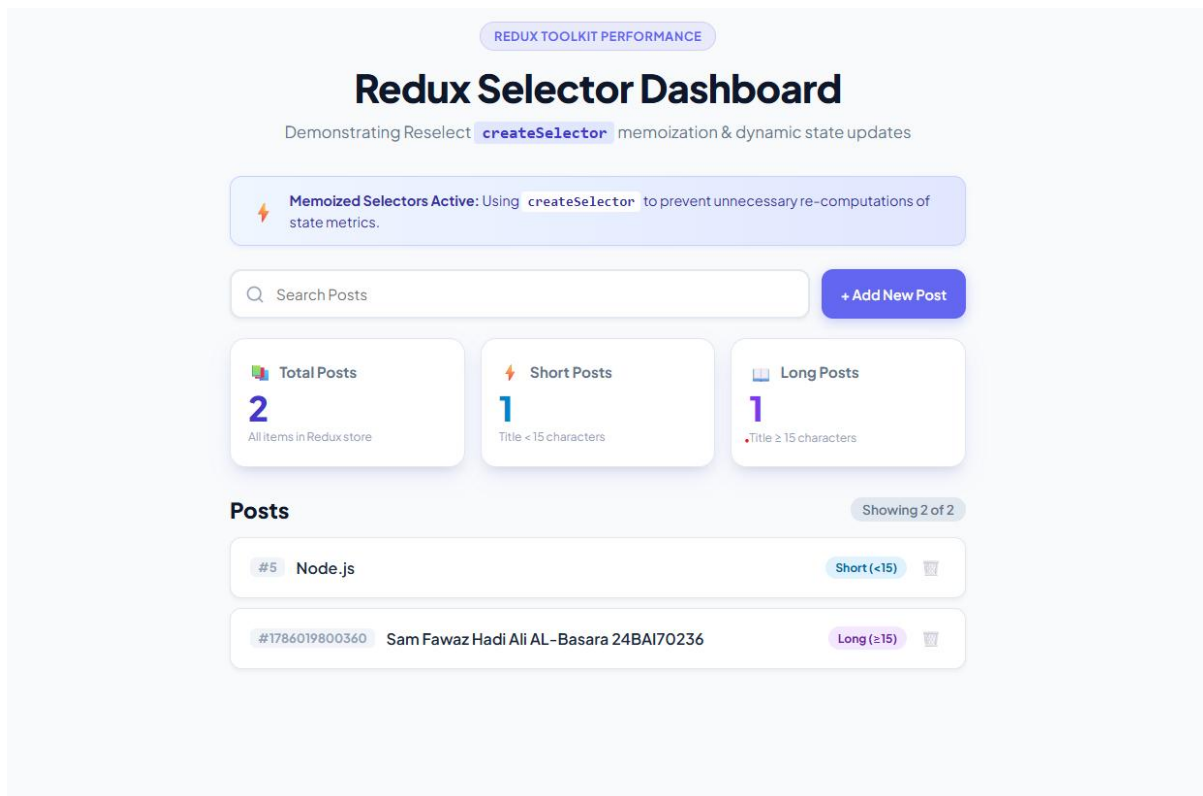


Fig: 2.2 Redux Selector Dashboard

8. LEARNING OUTCOMES

- Understand derived state using Redux Selectors.
- Implement memoized selectors using createSelector().
- Optimize state retrieval from the Redux Store.
- Reduce unnecessary recomputations using memoization.
- Filter and search data efficiently.
- Improve rendering performance in React applications.
- Develop scalable and performance-optimized frontend applications using Redux Toolkit.



EXPERIMENT – 3

NAME: Sam Fawaz Hadi AL-Basara
BRANCH: CSE-AIML (CS221)
SEMESTER: 5TH
SUBJECT: Full Stack - II

UID: 24BAI70236
SECTION/GROUP: 24AIT_NTPP2[A]
DATE OF PERFORMANCE: 24/07/2026
SUBJECT CODE: 24CSP-337

EXPERIMENT 3.1

1. AIM

To design and implement a secure authentication system using JWT for user login and session management.

2. OBJECTIVE

1. To understand authentication mechanisms in web applications
2. To implement token-based authentication using JWT
3. To manage user sessions in a stateless architecture
4. To handle token storage and validation securely

3. SOFTWARE REQUIREMENT

- React.js
- Node.js (optional/mock-based initially)
- JWT libraries (conceptual understanding)
- Browser (for testing)

4. THEORY

Authentication is the process of verifying the identity of a user before granting access to system resources. Traditional session-based authentication relies on server-side session storage, which can become inefficient in distributed systems.

JWT-based authentication introduces a **stateless approach**, where the server issues a signed token containing user information. This token is stored on the client and sent with each request for verification.

A JWT consists of three parts:

- Header – Defines the algorithm
- Payload – Contains user data (claims)
- Signature – Ensures integrity and authenticity

5. IMPLEMENTATION/PROCEDURE

1. Create login form UI
2. Validate user credentials (*mock or static*)
3. Generate or simulate JWT token
4. Store token (localStorage/sessionStorage)
5. Attach token to requests
6. Decode token to extract user information

6. CODE

Authentication Service (authService.js)

```
export const authenticateUser = (username, password) => {  
  const validUser = {  
    username: "Lovleen",
```

```
password: "E20454",
email: "Lovleen.e20454@cumail.in",
role: "Administrator",
};

if (
  username === validUser.username &&
  password === validUser.password
) {
  const header = btoa(
    JSON.stringify({
      alg: "HS256",
      typ: "JWT",
    })
  );

  const payload = btoa(
    JSON.stringify({
      username: validUser.username,
      email: validUser.email,
      role: validUser.role,
    })
  );

  const signature = btoa("jwt-signature");

  return `${header}.${payload}.${signature}`;
}

return null;
};
```

Login Authentication (Login.js)

```
const handleLogin = () => {
  const token = authenticateUser(username, password);

  if (token) {
    localStorage.setItem("token", token);
    login();
  } else {
    setError("Invalid Username or Password");
  }
};
```

Store JWT Token

```
localStorage.setItem("token", token);
```

Decode JWT Token (token.js)

```
export const getUserFromToken = () => {
  const token = localStorage.getItem("token");
```

```
if (!token) return null;

const payload = JSON.parse(
  atob(token.split(".")[1])
);

return payload;
};
```

Dashboard

```
const user = getUserFromToken();
```

```
<h2>{user?.username}</h2>
```

```
<p>{user?.email}</p>
```

```
<p>{user?.role}</p>
```

Logout

```
const logout = () => {
  localStorage.removeItem("token");
  setIsLoggedIn(false);
};
```

App Component

```
{
  isLoggedIn
    ? <Dashboard logout={logout}/>
    : <Login login={login}/>
}
```

7. OUTPUT

The JWT Authentication system was successfully implemented. Users were able to log in using valid credentials, after which a JWT token was generated and securely stored in the browser's localStorage. The application successfully decoded the token to retrieve user information and displayed it on the dashboard. Invalid login attempts were rejected with an appropriate error message, and the logout functionality removed the token, terminating the user session.

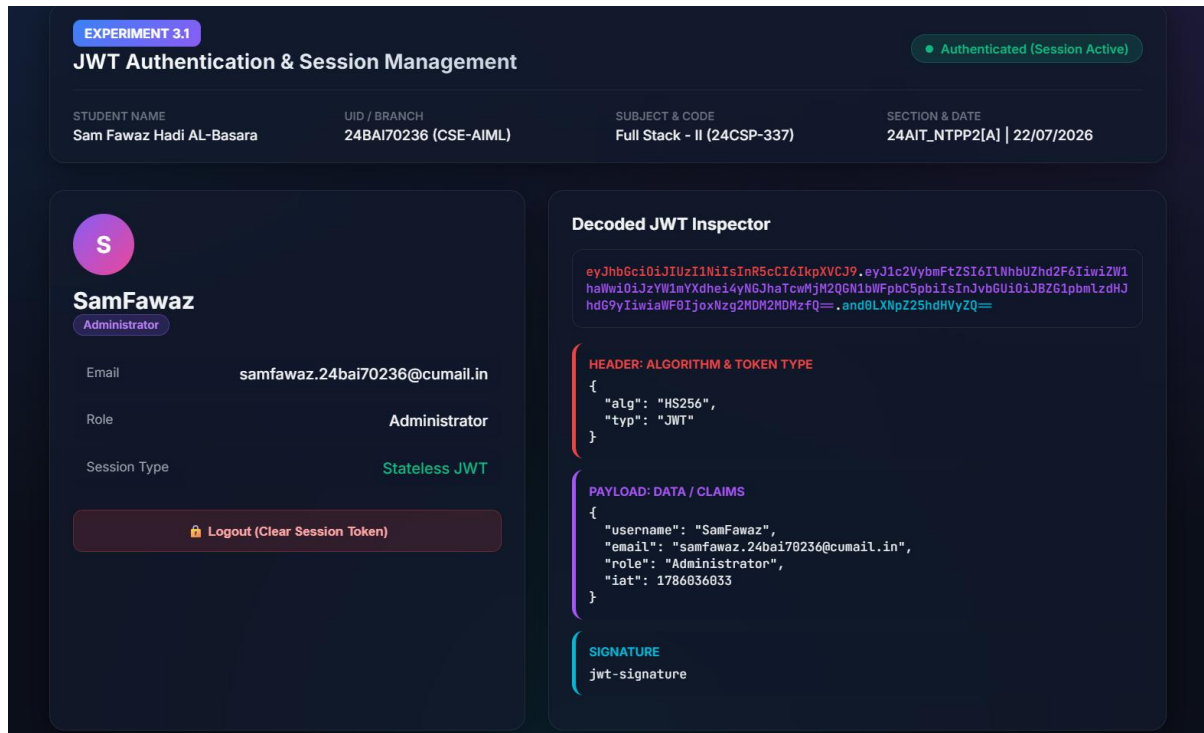


Fig. 3.1 JWT Authentication system

8. LEARNING OUTCOMES

- Understand the concept of authentication in web applications.
- Implement JWT-based authentication for secure user login.
- Generate and manage JWT tokens for stateless session handling.
- Store authentication tokens securely using localStorage.
- Decode JWT tokens to retrieve user information.
- Implement login, logout, and session management functionality.
- Develop secure authentication workflows in React applications.

EXPERIMENT 3.2

1. AIM

To implement role-based access control and secure application routes based on user permissions.

2. OBJECTIVE

1. To understand authorization mechanisms in applications
2. To implement RBAC for access control
3. To protect routes based on user roles
4. To dynamically render UI elements based on permissions

3. SOFTWARE REQUIREMENT

1. React.js
2. React Router
3. State management (Context API / Redux)

4. THEORY

While authentication verifies user identity, **authorization** determines what actions a user is permitted to perform. Role-Based Access Control (RBAC) is a widely used model where users are assigned roles, and each role has defined permissions.

Common roles include:

- Admin
- Editor
- Viewer

RBAC ensures:

- Controlled access to sensitive features
- Simplified permission management
- Improved system security

Route protection is implemented to restrict navigation to certain pages unless the user has appropriate permissions.

5. IMPLEMENTATION/PROCEDURE

1. Define user roles (Admin, Editor, Viewer)
2. Store role in authentication state
3. Implement protected routes using React Router
4. Restrict access based on role
5. Conditionally render UI elements (buttons, pages)
6. Redirect unauthorized users

6. CODE

User Authentication (authService.js)

```
const users = [  
  {  
    username: "admin",  
    password: "admin123",  
    role: "Admin",  
  },  
  {  
    username: "editor",  
    password: "editor123",  
  },  
]
```

```
    role: "Editor",
  },
  {
    username: "viewer",
    password: "viewer123",
    role: "Viewer",
  },
];

export const authenticateUser = (username, password) => {
  const user = users.find(
    (u) =>
      u.username === username &&
      u.password === password
  );

  if (!user) return null;

  const header = btoa(
    JSON.stringify({
      alg: "HS256",
      typ: "JWT",
    })
  );

  const payload = btoa(
    JSON.stringify({
      username: user.username,
      role: user.role,
    })
  );

  const signature = btoa("rbac-secret");

  return `${header}.${payload}.${signature}`;
};
```

Login Authentication (Login.js)

```
const handleLogin = () => {

  const token = authenticateUser(
    username,
    password
  );

  if (!token) {
    setError("Invalid Username or Password");
    return;
  }
}
```



```
localStorage.setItem("token", token);
```

```
const payload = JSON.parse(  
  atob(token.split(".")[1])  
);
```

```
if (payload.role === "Admin")  
  navigate("/admin");
```

```
else if (payload.role === "Editor")  
  navigate("/editor");
```

```
else  
  navigate("/viewer");
```

```
};
```

Token Utilities (token.js)

```
export const getUser = () => {
```

```
  const token = localStorage.getItem("token");
```

```
  if (!token) return null;
```

```
  return JSON.parse(  
    atob(token.split(".")[1])  
  );
```

```
};
```

```
export const getRole = () => {
```

```
  const user = getUser();
```

```
  return user?.role;
```

```
};
```

```
export const logout = () => {
```

```
  localStorage.removeItem("token");
```

```
};
```

Protected Route (ProtectedRoute.js)

```
function ProtectedRoute({  
  children,  
  allowedRoles,
```

```
  }) {  
  
    if (!isAuthenticated()) {  
      return <Navigate to="/" />;  
    }  
  
    const role = getRole();  
  
    if (!allowedRoles.includes(role)) {  
      return <Navigate to="/unauthorized" />;  
    }  
  
    return children;  
  }  
}
```

Route Configuration (App.js)

```
<BrowserRouter>
```

```
  <Routes>
```

```
    <Route  
      path="/"  
      element={<Login />}  
    />
```

```
    <Route  
      path="/admin"  
      element={  
        <ProtectedRoute  
          allowedRoles={["Admin"]}  
        >  
          <Admin />  
        </ProtectedRoute>  
      }  
    />
```

```
    <Route  
      path="/editor"  
      element={  
        <ProtectedRoute  
          allowedRoles={["Editor"]}  
        >  
          <Editor />  
        </ProtectedRoute>  
      }  
    />
```

```
    <Route
```

```
path="/viewer"
element={
  <ProtectedRoute
    allowedRoles={["Viewer"]}
  >
    <Viewer />
  </ProtectedRoute>
}
/>

<Route
  path="/unauthorized"
  element={<Unauthorized />}
/>
</Routes>
</BrowserRouter>
```

Role-Based Dashboards

Admin Dashboard

- User Management
- Reports
- Settings
- Logout

Editor Dashboard

- Manage Content
- Edit Articles
- View Reports
- Logout

Viewer Dashboard

- View Content
- View Reports
- Profile
- Logout

Navigation Bar

<Navbar />

User Name

User Role

Logout Button

7. OUTPUT

The Role-Based Access Control (RBAC) system was successfully implemented. Users were authenticated using their credentials and redirected to dashboards based on their assigned roles (Admin, Editor, or Viewer). Protected routes restricted unauthorized access, and users attempting to access restricted pages were redirected to the Unauthorized (403 Access Denied) page. The application dynamically displayed different interfaces according to user permissions, demonstrating secure authorization and role-based navigation.

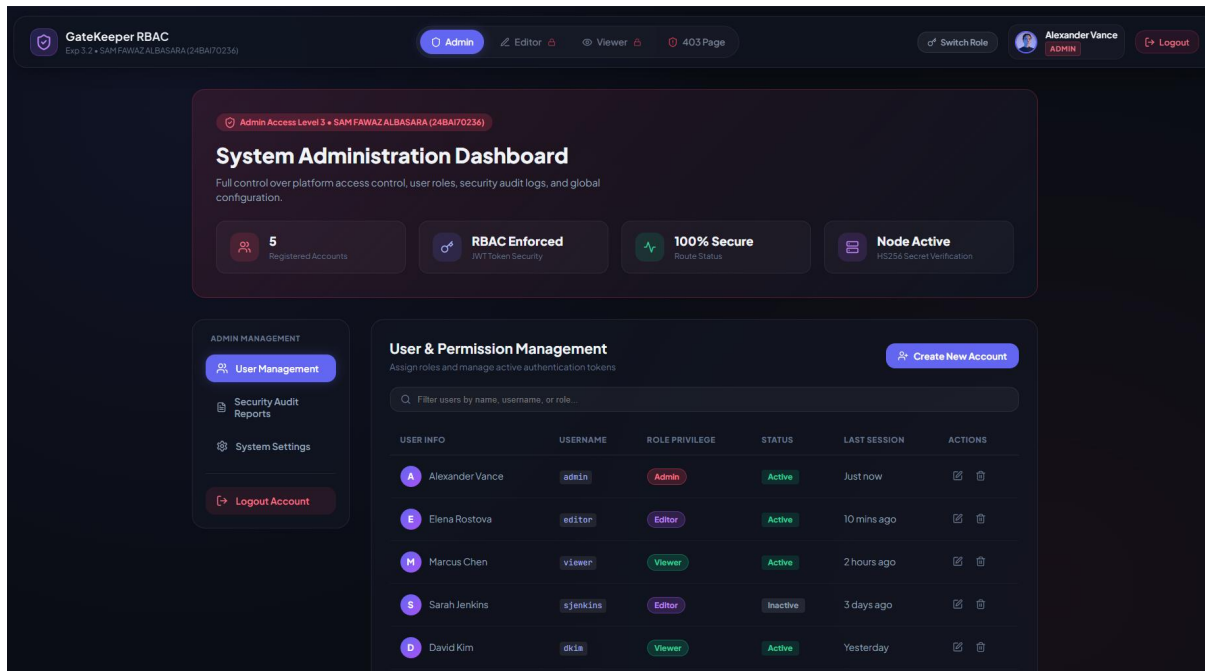


Fig:3.2 Role-Based Access Control (RBAC) system

8. LEARNING OUTCOMES

- Understand the concept of authorization and Role-Based Access Control (RBAC).
- Implement protected routes using React Router.
- Restrict application access based on user roles.
- Dynamically render user interfaces according to permissions.
- Manage user roles and secure navigation in React applications.
- Implement role-based authentication and authorization for enhanced application security.
- Develop secure, scalable, and permission-based web applications.



EXPERIMENT – 4

NAME: Sam Fawaz Hadi Ali AL-Basara
BRANCH: CSE-AIML (CS221)
SEMESTER: 5TH
SUBJECT: Full Stack - II

UID: 24BAI70236
SECTION/GROUP: 24AIT_NTPP 2[A]
DATE OF PERFORMANCE: 31/07/2026
SUBJECT CODE: 24CSP-337

EXPERIMENT 4.1

1. AIM

To design and implement an interactive calendar interface for scheduling and managing posts.

2. OBJECTIVE

1. To understand time-based data visualization in UI systems
2. To implement calendar-based scheduling interfaces
3. To map structured data to temporal layouts
4. To enable user interactions such as drag-and-drop

3. SOFTWARE REQUIREMENT

- React.js
- Calendar library (e.g., FullCalendar / custom)
- Date utility libraries (optional)
- Code Editor

4. THEORY

Calendars are essential UI components for representing time-based data. In applications such as social media schedulers, events (posts) must be mapped to specific dates and time slots. A calendar system involves:

- **Temporal data modeling** (date, time, duration)
- **Event mapping** (linking posts to time slots)
- **User interaction handling** (click, drag, resize)

Interactive calendars enhance usability by allowing users to:

- Visually organize content
- Modify schedules dynamically
- Manage large datasets efficiently

Academic Insight:

Calendar systems fall under **Human-Computer Interaction (HCI)** and **Information Visualization**, where data is presented in a form that supports intuitive understanding and interaction.

5. IMPLEMENTATION/PROCEDURE

1. Design calendar UI layout (day/week/month view)
2. Map post data to calendar events
3. Render events dynamically
4. Implement click interactions (view/edit)
5. Add drag-and-drop scheduling (*optional/advanced*)
6. Sync calendar with application state

6. CODE

Import FullCalendar

```
import FullCalendar from "@fullcalendar/react";  
import dayGridPlugin from "@fullcalendar/daygrid";  
import interactionPlugin from "@fullcalendar/interaction";
```

Sample Event Data

```
const events = [  
  {  
    id: "1",  
    title: "Instagram Post",  
    date: "2026-08-05",  
  },  
  {  
    id: "2",  
    title: "LinkedIn Post",  
    date: "2026-08-08",  
  },  
  {  
    id: "3",  
    title: "Facebook Campaign",  
    date: "2026-08-12",  
  },  
];
```

Render Calendar

```
<FullCalendar  
  plugins={[dayGridPlugin, interactionPlugin]}  
  initialView="dayGridMonth"  
  editable={true}  
  selectable={true}  
  events={events}  
>
```

Handle Event Click

```
const handleEventClick = (info) => {  
  alert(  
    "Selected Event: " + info.event.title  
  );  
};  
<FullCalendar  
  eventClick={handleEventClick}  
>
```

Handle Drag and Drop

```
const handleEventDrop = (info) => {  
  alert(  
    `${info.event.title} moved to ${info.event.start.toDateString()}`  
  );  
};  
<FullCalendar  
  eventDrop={handleEventDrop}  
>
```

Calendar Component

```
function CalendarView() {  
  
  return (  
  
    <FullCalendar  
  
      plugins={[  
        dayGridPlugin,  
        interactionPlugin,  
      ]}  
  
      initialView="dayGridMonth"  
  
      events={events}  
  
      editable={true}  
  
      selectable={true}  
  
      eventClick={handleEventClick}  
  
      eventDrop={handleEventDrop}  
  
    />  
  
  );  
  
}
```

Main Application

```
import CalendarView from "../components/CalendarView";  
  
function App() {  
  
  return (  
  
    <div>  
  
      <h1>Post Scheduling Calendar</h1>  
  
      <CalendarView />  
  
    </div>  
  
  );  
  
}  
  
export default App;
```

7. OUTPUT

The interactive calendar scheduling system was successfully implemented using React and the FullCalendar library. Posts were displayed as calendar events mapped to their respective dates, allowing users to visualize scheduled content in a monthly calendar view. The application supported event interactions such as viewing event details on click and rescheduling posts through drag-and-drop functionality, providing an efficient interface for content planning and schedule management.

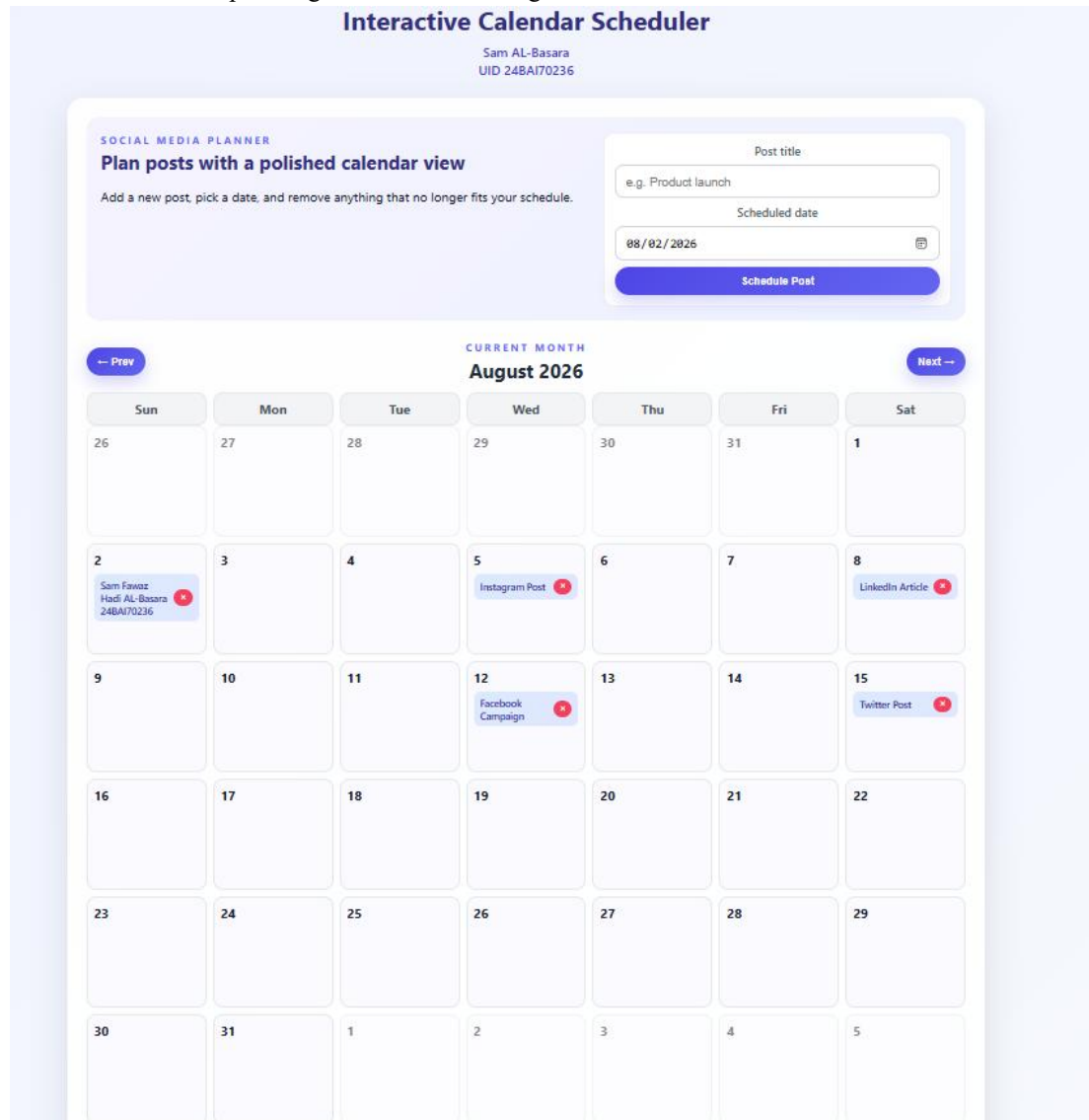


Fig 4.1 Interactive calendar scheduling system

8. LEARNING OUTCOMES

- Understand the concept of calendar-based scheduling in web applications.
- Design and implement an interactive calendar interface using React.
- Map structured post data to calendar events and time slots.
- Implement event handling for viewing and managing scheduled posts.
- Apply drag-and-drop functionality for dynamic event rescheduling.
- Synchronize calendar events with application state for efficient data management.
- Develop user-friendly scheduling systems for content planning and time-based applications.

EXPERIMENT 4.2

1. AIM

To optimize rendering performance and implement testing strategies for interactive UI components.

2. OBJECTIVE

1. To understand performance bottlenecks in UI systems.
2. To optimize rendering using memoization techniques.
3. To reduce unnecessary re-renders in React applications.
4. To implement testing for UI components and application logic.

3. SOFTWARE REQUIREMENT

1. React.js
2. Jest--
3. React Testing Library
4. Browser DevTools
5. Visual Studio Code
6. Node.js & npm

4. THEORY

Interactive React applications such as calendars frequently update their UI based on user interactions. Without optimization, repeated rendering of components can negatively affect application performance and responsiveness.

Performance optimization techniques such as `*React.memo*`, `**useMemo`, and `**useCallback*` help reduce unnecessary component re-renders and improve rendering efficiency. Memoization stores previously computed values and function references, ensuring computations occur only when required.

Testing is an essential part of frontend development. Unit testing verifies that UI components and application logic behave as expected under different scenarios, improving software reliability, maintainability, and stability.

By combining rendering optimization and testing, developers can build scalable, responsive, and reliable React applications.

5. IMPLEMENTATION/PROCEDURE

1. Create a React application and install the required dependencies.
2. Develop reusable React components for the calendar interface.
3. Identify components that re-render frequently.
4. Apply `React.memo()` to prevent unnecessary component re-rendering.
5. Use `useMemo()` to memoize expensive computations.
6. Use `useCallback()` to memoize event handler functions.
7. Optimize state updates to improve rendering performance.
8. Install Jest and React Testing Library.
9. Write unit test cases for calendar components.
10. Test user interactions such as event selection, drag-and-drop, and updates.
11. Execute test cases and verify successful execution.
12. Run the application and observe improved rendering performance.

6. CODE

Optimize Component using React.memo

```
javascript
const Calendar = React.memo(function Calendar(props) {
  return (
    <div>
      <h2>Post Calendar</h2>
    </div>
  );
});
```

Memoize Expensive Computation

```
javascript
const filteredEvents = useMemo(() => {
  return events.filter(
    (event) => event.title.includes(search)
  );
}, [events, search]);
```

Memoize Event Handler

```
javascript
const handleEventClick = useCallback((event) => {
  alert(event.title);
}, []);
```

Component Rendering

```
javascript
<Calendar
  events={filteredEvents}
  onEventClick={handleEventClick}
/>
```

Unit Test

```
javascript
import { render, screen } from "@testing-library/react";
import Calendar from "../Calendar";

test("renders calendar heading", () => {
  render(<Calendar />);
  expect(
    screen.getByText("Post Calendar")
  ).toBeInTheDocument();
});
```

Run Test

bash
npm test

Files Used

- App.js
- Calendar.js
- Calendar.test.js
- App.css

7. OUTPUT

The calendar application was successfully optimized using React.memo, useMemo, and useCallback, resulting in reduced unnecessary component re-renders and improved rendering performance. Unit tests executed successfully using Jest and React Testing Library, verifying the correctness of UI components and user interactions. The optimized application provided a faster, more responsive, and reliable user experience.

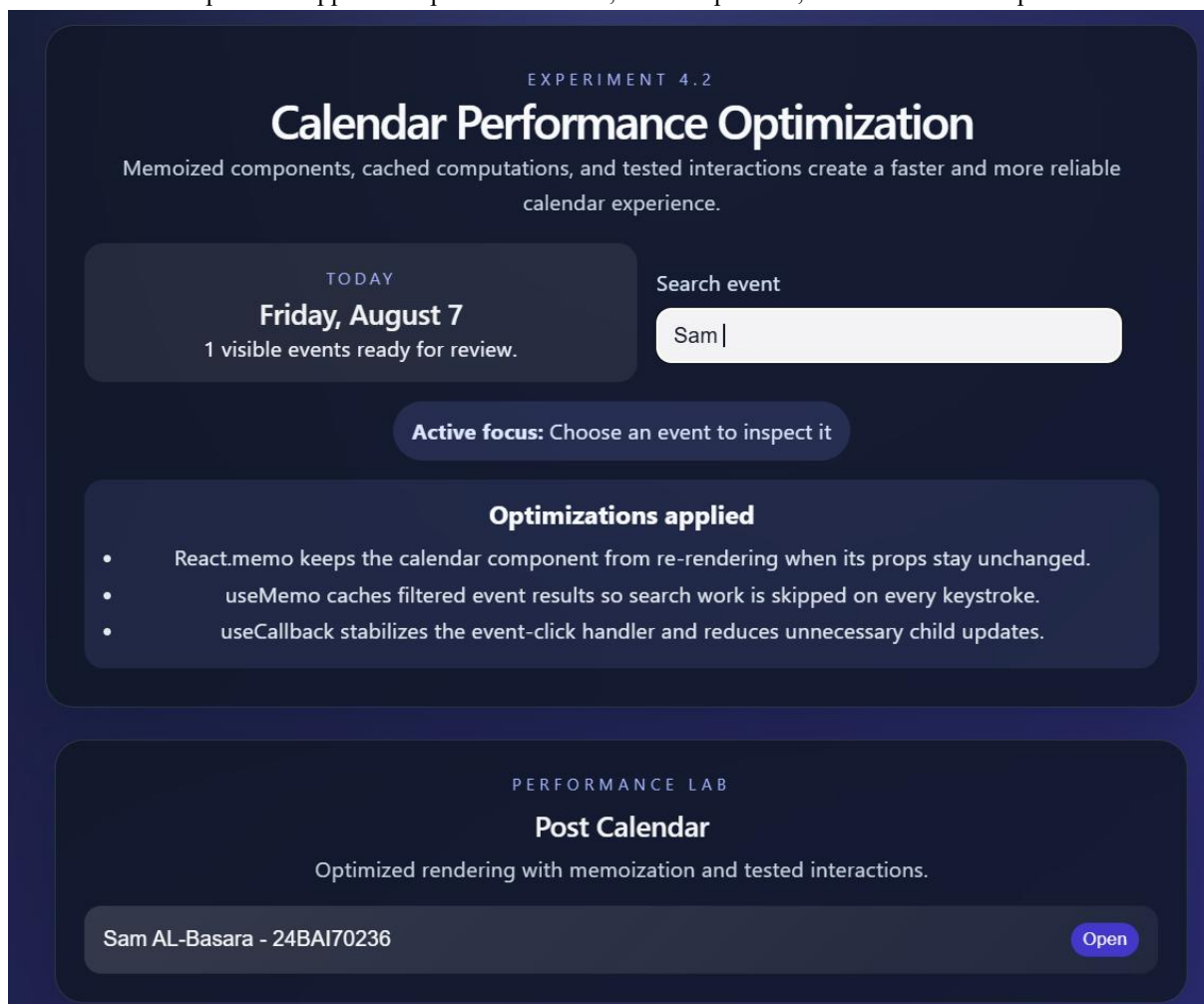


Fig 4.2: Calendar Application

8. LEARNING OUTCOMES

- Understand rendering performance optimization techniques in React.
- Implement memoization using *React.memo, **useMemo, and **useCallback*.



- Reduce unnecessary component re-renders to improve application efficiency.
- Optimize state updates for interactive UI components.
- Develop and execute unit tests using Jest and React Testing Library.
- Validate component functionality and user interactions through testing.
- Build scalable, optimized, and maintainable React applications following performance engineering and software testing principles.